THE WHOLE POINT OF LINKED LISTS

PART TWO

When the Map Refuses to Stay Still

An illustrated journey through singly linked-list operations



Lijo M. Jose

Assistant Professor, School of Computer Science and Engineering

In Part One, we followed the mission, encountered its central problem, and watched Captain Caitlyn arrive at an unusual solution. Instead of predicting the entire route in advance, each beacon would remember only the destination that followed it.

That idea translated beautifully into the computing world as a linked list.

But building a route is only the beginning.

Galaxies change. Planets become available. New discoveries create useful detours. Friendly civilizations share their paths. Black holes and cosmic storms make previously safe worlds impossible to visit.

A useful map cannot merely remember yesterday's universe. It must adjust to today's universe without losing the journey it already knows.

Let us see whether Caitlyn's chain of beacons is ready for that.



Caitlyn studies a route that will have to survive a changing universe.

1. The Journey Begins Again

For eleven months, the Explorer Bot had done exactly what Mission Control asked of it.

It had left home with no complete map, accepted one available world at a time, installed a beacon, and carried the newest coordinate back to the beacon it had left behind. The route had grown slowly behind it until the final scheduled survey was complete.

At the last planet, the bot installed one final beacon. It stored the planet's observations, left the coordinate compartment as `NULL`, and transmitted its closing report.

There was no next destination to record. The mission was over.

The bot was decommissioned there.

Not destroyed. Not lost. Simply powered down after a job well done.

For a while, the last `NULL` was perfectly correct.

Then the long-range observatory found something beyond it.

A faint pattern of radio bursts appeared farther into the unexplored region. At first the signal looked like noise. Then it repeated. The intervals were too precise to be natural, and the scientific council reopened the expedition.

A new bot was prepared. Its starting point would be the final planet of the old mission. From there, it would continue into the unknown.

The new bot landed on the old final world, activated the silent beacon, and travelled onward. Several days later it found a stable planet with accessible water, a manageable atmosphere, and no claim from another civilization. It installed a fresh beacon there.

Now look at the two routes Caitlyn has in front of her.

The old route still begins at `head` and ends at the decommissioned bot's planet. Its final beacon points to `NULL`.

The new planet also has a beacon. Because nothing has yet been discovered beyond it, that beacon also points to `NULL`.

Two correct pieces exist.

They are simply not one route.

> **Pause at Mission Control**
> How would you make the new planet continue the old journey? Which beacon must change, and how can Caitlyn reach it if Mission Control has stored only `head`?

The new beacon cannot repair this by pointing backwards to the old final planet. Caitlyn follows coordinates forward. Reversing this single arrow would let the new planet find the old one, but it would not let anyone beginning at home find the new planet.

Perhaps `head` should point directly to the new beacon.

That would certainly make the new planet easy to reach. Unfortunately, it would also abandon every planet discovered by the first expedition. The old route would remain in memory, but Mission Control would no longer know its entrance.

The link that must change belongs to the old final beacon. It is the beacon whose `next` compartment currently contains `NULL`.

So Caitlyn must begin at `head` and follow the chain until she finds a beacon whose `next` is `NULL`. She should stop there - at the last live node, not one step after it.

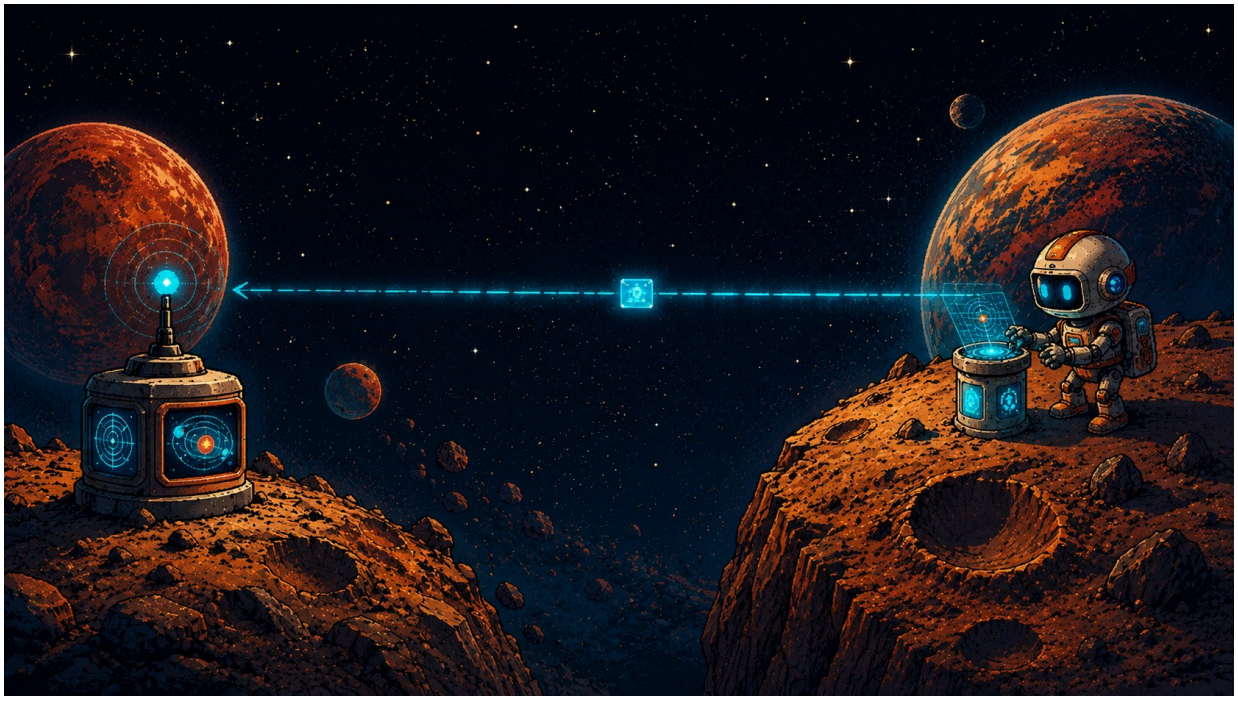
Only then can she replace that final `NULL` with the address of the new beacon.

The new beacon remains unchanged. Its own `next` is already `NULL`, because it is now the final known destination.

The journey has begun again.

> **What have we just discovered?**
> The mission has performed **insertion at the end of a singly linked list**, also called appending a node.

Computing Translation



The old final beacon must learn the address of the newly discovered world.

The operation has three phases:

1. Allocate and initialize a standalone node whose `next` is `NULL`.
2. If the list is not empty, traverse until `current->next == NULL`.
3. Store the new node's address in the last node's `next` field.

```
Node *insertBack(Node *head, int value)
{
    Node *newNode = createNode(value);

    if (head == NULL)
        return newNode;

    Node *current = head;
    while (current->next != NULL)
        current = current->next;

    current->next = newNode;
    return head;
}
```

Notice the loop condition. We do not continue while `current != NULL`. Doing that would move `current` one step beyond the last node, leaving it equal to `NULL`. A null pointer has no `next` field through which a new node can be attached.

We stop while we still own a real beacon.

The empty route needs one small decision. If `head == NULL`, there is no final node to repair. The newly allocated node is both the first and the last node, so the function returns it as the new `head`.

With only `head`, appending requires a full traversal and therefore takes `O(n)` time. Creating and linking the node uses `O(1)` additional space.

Must the route remember its last beacon?

Mission Control may decide to maintain another pointer named `tail`, always pointing to the final node. Appending can then be done in `O(1)` time.

That speed is not free. Every operation that changes the last node must also keep `tail` correct. A singly linked list does not magically contain a tail pointer; the program chooses whether the additional promise is worth maintaining.

> **Before you leave this section**
> If Mission Control expects the expedition to append thousands of planets but rarely deletes anything, would you keep only `head`, or both `head` and `tail`? What new mistakes become possible after adding `tail`?

2. The World the Aliens Left Behind

The renewed expedition continued outward, but the next change came from the opposite direction.

Close to Caitlyn's home world was a small planet called **Ephyra**. It was nearer than the first planet in the existing route and had excellent communication range. Years earlier, however, a trading civilization called the Velari had established a colony there. Ephyra was useful, but it was not available.

The Velari eventually discovered that useful and affordable were not the same thing.

Ephyra's storms damaged their surface equipment every few months. Maintaining the colony cost more than the trade route earned. One morning, Caitlyn received a courteous transmission: the Velari were leaving.

She contacted them before their final transport departed.

"Would you object if our expedition took over the site?"

The answer arrived almost immediately.

"Please do. And if you can make the weather behave, you may keep that too."

The weather was not included in the agreement, but the planet was.

Ephyra was too valuable to ignore. Because it was the closest safe stop to home, Caitlyn did not want to place it at the far end of the route. Every future traveller should visit it first.

She installed a new beacon there. It contained Ephyra's observations and, like every newly created beacon, its route compartment initially contained `NULL`.

Now Caitlyn had one new beacon and one complete old route.

The old route still began at `head`.

> **Pause at Mission Control**
> How can Ephyra become the new entry point without making the old entry - and every planet after it - disappear?

The most tempting answer is also the shortest:

```
head = newNode;
```

After all, `head` is supposed to identify the first node, and Ephyra is supposed to become first.

The assignment achieves exactly that.

It also creates a rather lonely route.

`head` now reaches Ephyra. Ephyra's beacon still points to `NULL`. The original first beacon is somewhere in memory, and so is every beacon after it, but the only address Mission Control kept for entering that chain has just been overwritten.

The planets did not vanish.

The directions did.

Let us return to the moment before `head` changed. At that moment Caitlyn knows two addresses:

- `newNode`, the address of Ephyra's beacon;
- `head`, the address of the old first beacon.

Which address is in danger?

The old `head`.

So Ephyra's beacon must preserve it first:

```
newNode->next = head;
```

Nothing has been published yet. The existing route still begins at `head`, and Ephyra now knows how to enter it.

Only after that connection exists may Mission Control change the official entrance:

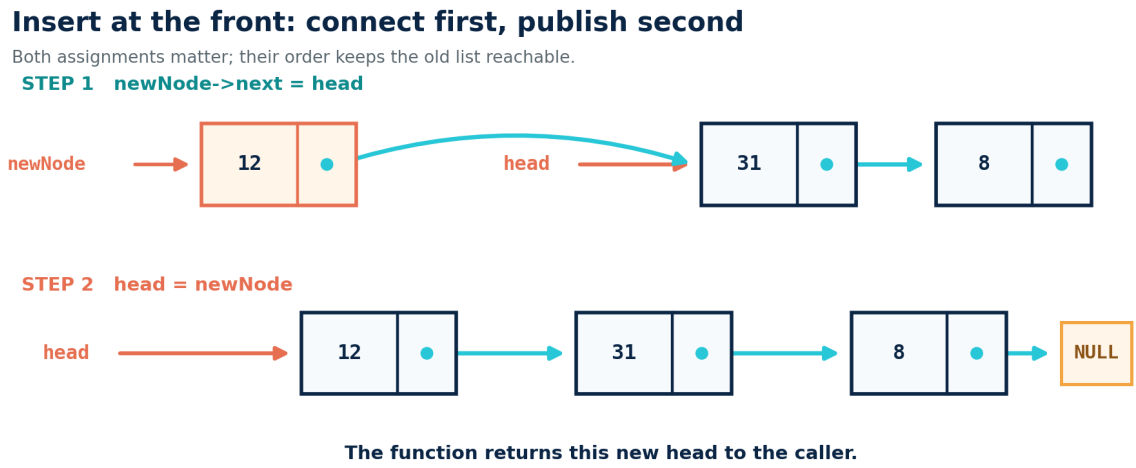
```
head = newNode;
```

Now a traveller begins at Ephyra, reads its beacon, and continues to the route that already existed.

Connect first. Publish second.

> **What have we just discovered?**
> Caitlyn has performed **insertion at the beginning of a singly linked list**. The new node is connected to the old list before it becomes the new `head`.

Computing Translation



Connect the new node to the old head before returning it as the new head.

```
Node *insertFront(Node *head, int value)
{
    Node *newNode = createNode(value);

    newNode->next = head;
    return newNode;
}
```

There is no special branch for an empty list. If `head` is already `NULL`, the assignment `newNode->next = head` simply leaves the new node pointing to `NULL`. Returning `newNode` creates a valid one-node list.

The caller must store the returned address:

```
head = insertFront(head, value);
```

The function receives a copy of the caller's pointer. Returning the new address is how our main implementation makes a changed entry point visible to the caller. A separate technical note near the end of the chapter will show the double-pointer alternative; Caitlyn's main route will continue using returned heads.

Insertion at the beginning changes a fixed number of pointers, so it takes `O(1)` time and `O(1)` additional space.

> **A small change to the agreement**
> Suppose the Velari had left three neighbouring worlds, already connected by their own beacons. How would you attach the entire three-world chain before the current `head`? Which address would the last Velari beacon need?

3. The Moon Caitlyn Refused to Ignore

By the time Caitlyn reached **Orialis**, she had already travelled the route often enough to recognize each beacon's signal before seeing the planet itself.

Orialis was a quiet blue world whose upper atmosphere produced remarkable electrical storms. The bot had recorded the storms, installed its beacon, and continued to **Tavros**, the next planet in the route.

The route was therefore perfectly ordinary:

```
... -> Orialis -> Tavros -> ...
```

Which, in Caitlyn's experience, meant it was about to become inconvenient.

While calibrating her instruments, she detected a repeating magnetic pulse from one of Orialis's moons. The moon was small enough to have escaped the bot's original survey. Its name in the old astronomical catalogue was **Nysa**.

Caitlyn took a short flight to investigate.

Nysa's surface contained a natural magnetic lattice capable of stabilizing long-range sensors. A station there could improve the entire expedition's communication accuracy. Ignoring it merely because the bot had passed Orialis months earlier would be absurd.

So Caitlyn installed a beacon on Nysa.

The future route should now become:

```
... -> Orialis -> Nysa -> Tavros -> ...
```

Nysa's beacon was initialized and waiting. Orialis still pointed directly to Tavros.

> **Pause at Mission Control**
> Orialis owns the coordinate of Tavros. Caitlyn must make Orialis point to Nysa. If she replaces that coordinate immediately, where will the address of Tavros survive?

The obvious assignment is:

```
current->next = newNode;
```

Here `current` points to Orialis and `newNode` points to Nysa.

Perform the assignment and examine the route.

Orialis now points to Nysa. Good.

Nysa still points to `NULL`. Not so good.

Where is Tavros?

It still exists. So do all the planets after it. But the address that connected Orialis to that suffix used to live in `current->next`, and Caitlyn has overwritten it.

Perhaps she can repair Nysa afterward:

```
newNode->next = current->next;
```

Look carefully before accepting that line. What does `current->next` contain now?

It no longer contains Tavros. It contains `newNode`.

Nysa therefore points to itself. Caitlyn has created a one-moon circular tour while Tavros and the rest of the mission drift beyond reach.

The code did exactly what she asked.

She simply asked in the wrong order.

Let us replay the moment before any link changed.

At that moment:

- `current` points to Orialis;
- `current->next` contains Tavros's address;
- `newNode` points to Nysa;
- `newNode->next` is `NULL`.

Which address is in danger?

Tavros's address.

Then that is the address that must survive first.

```
newNode->next = current->next;
```

Nysa now knows how to continue to Tavros. The original route is still intact because Orialis still points to Tavros as well.

Only now may Caitlyn redirect Orialis:

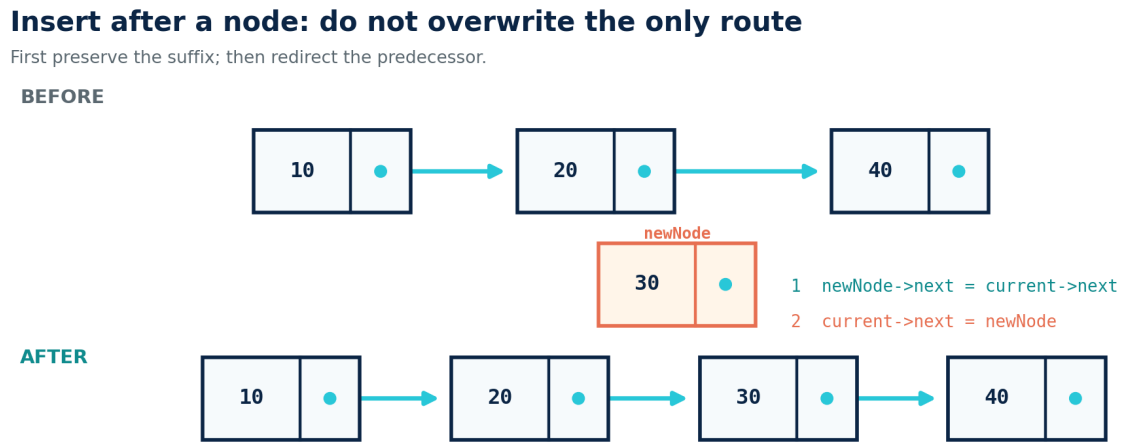
```
current->next = newNode;
```

The detour is complete.

Nothing moved in memory. One endangered address was preserved, and one existing link was redirected.

> **What have we just discovered?**
> This is **insertion after a specified node** in a singly linked list. The old suffix must be saved in the new node before the predecessor's link is overwritten.

Computing Translation



The old suffix is preserved before the predecessor is redirected.

```
Node *insertAfter(Node *head, int key, int value)
{
    Node *current = head;

    while (current != NULL && current->data != key)
        current = current->next;

    if (current == NULL) {
        printf("%d not found; list unchanged.\n", key);
        return head;
    }

    Node *newNode = createNode(value);
    newNode->next = current->next;
    current->next = newNode;
    return head;
}
```

The function first searches for the node containing `key`. Allocation happens only after that node is found. This is deliberate: if the key is absent, no new node needs to be created and then discarded.

If `current` is the final node, `current->next` is `NULL`. The same two assignments append the new node correctly; no special final-node branch is needed.

Searching may examine every node, so the operation takes `O(n)` time in the worst case. Once `current` is known, the actual insertion takes `O(1)` time.

> **What if Nysa were only the beginning?**
> Suppose Caitlyn discovers that Nysa already leads through three connected moons before returning to Tavros. She must insert an entire linked route after Orialis, not a single node. Which two links must change? We will return to this question when another civilization shares a complete trail.

4. The Voyage That Was Too Long

The route between **Kaelus** and **Virelia** looked manageable on the map.

Maps are very good at making empty space look harmless.

The first journey took Caitlyn nearly thirty hours. There was nowhere to refuel, nowhere to perform repairs, and nowhere to rest without leaving the mapped corridor. By the time Virelia's beacon appeared on her scanner, even the ship's navigation assistant had stopped making cheerful suggestions.

On the return journey, Caitlyn detected a large station outside the route. It belonged to the **Aruun**, a friendly species whose traders had occasionally contacted Mission Control. The station was called **Haven-9**.

Haven-9 had fuel, repair docks, food, and - most importantly at that moment - beds.

The Aruun agreed to let Caitlyn's expedition use it as a permanent rest stop.

The route should no longer travel directly from Kaelus to Virelia. Haven-9 should come immediately before Virelia:

```
... -> Kaelus -> Haven-9 -> Virelia -> ...
```

Caitlyn knew Virelia's beacon. She could search the route and find it.

Then she noticed the awkward part.

> **Pause at Mission Control**
> The link that must change is not stored in Virelia. It is stored in the beacon that currently points to Virelia. Can a singly linked route ask Virelia to reveal its previous planet?

It cannot.

Virelia stores only one route coordinate: the planet that follows it. Nothing inside Virelia points backward to Kaelus.

Caitlyn could first find Virelia and then begin again from `head`, searching for a node whose `next` equals Virelia. That would work, but it would traverse part of the route twice.

A better question guides the first traversal:

> Which node must I stop at to perform the insertion?

Not Virelia.

Its predecessor.

So Caitlyn begins at `head` and examines `current->next`. She continues while the next node is not Virelia. When `current->next` finally contains Virelia's address, `current` is the beacon whose link must be repaired.

Now the familiar order returns:

4. Haven-9 first remembers Virelia.
5. The predecessor then points to Haven-9.

The route gains a rest stop without losing its destination.

There is one special possibility. What if Mission Control asks to insert a station before the current first node? No predecessor exists. That is exactly the Ephyra problem from the previous section: insertion at the beginning.

> **What have we just discovered?**
> This is **insertion before a specified node**. In a singly linked list, the operation normally searches for the target's predecessor because the target has no backward link.

Computing Translation

```
Node *insertBefore(Node *head, int key, int value)
{
    if (head == NULL) {
        printf("List is empty; %d not found.\n", key);
        return NULL;
    }

    if (head->data == key)
        return insertFront(head, value);

    Node *previous = head;
    Node *current = head->next;

    while (current != NULL && current->data != key) {
        previous = current;
        current = current->next;
    }

    if (current == NULL) {
        printf("%d not found; list unchanged.\n", key);
        return head;
    }

    Node *newNode = createNode(value);
    newNode->next = current;
    previous->next = newNode;
    return head;
}
```

`previous` and `current` move together. The invariant during the traversal is simple:

```
previous->next == current
```

When the target is found, the new node first points to `current`, preserving the target and its suffix. Then `previous->next` is redirected to the new node.

The search makes the operation `O(n)` in the worst case. The link repair itself remains `O(1)`.

> **A clue for a later route**
> Haven-9 cannot tell Caitlyn where she came from. What extra information would every beacon need if travellers frequently wanted to move directly to the previous world?

5. Before the Fiftieth Stop

The next request arrived without a planet name.

Mission Control had studied fuel use across the expanded route and found a weakness. Travellers needed a maintenance station before the fiftieth stop.

The instruction sounded precise:

> Install the new station immediately before stop 50.

It was precise in one sense and surprisingly unhelpful in another.

Caitlyn had not been given the planet's name. She had not been given its address. She had not even been given the name of the planet before it.

She had been given a number.

If this route were an array, she might compute the address of position 50 from the address of position 1. But the beacons are scattered across whatever locations were available when they were allocated. There is no meaningful `head + 49` journey.

> **Pause at Mission Control**
> If the structure has no direct jump to the fiftieth node, where must Caitlyn begin, what must she count, and at which node should she stop?

She begins at `head`.

But should she stop at position 50?

Recall Haven-9. To insert before a node, Caitlyn needs the predecessor. For an insertion at position 50, she must stop at position 49.

That observation removes much of the confusion. Many positional insertion bugs are not really arithmetic errors. They come from failing to state which node the traversal is supposed to reach.

We will number positions from 1.

- Inserting at position 1 means placing the new node before the current `head`.
- Inserting at position 2 means stopping at the current position 1.
- Inserting at position `k` means stopping at position `k - 1`.
- Inserting at position `length + 1` appends after the current last node.

What if Caitlyn reaches `NULL` before she reaches position 49?

Then stop 50 does not exist, and the request cannot be completed. Silently attaching the station somewhere else would not be a helpful interpretation of Mission Control's instruction.

> **What have we just discovered?**
> This is **insertion at a specified position** in a singly linked list. The traversal is designed to stop at the node immediately before the requested position.

Computing Translation

```
Node *insertAt(Node *head, int position, int value)
{
    if (position < 1) {
        printf("Position must be at least 1.\n");
        return head;
    }

    if (position == 1)
        return insertFront(head, value);

    Node *previous = head;
    for (int i = 1;
         i < position - 1 && previous != NULL;
         ++i)
        previous = previous->next;

    if (previous == NULL) {
        printf("Position exceeds list length + 1.\n");
        return head;
    }

    Node *newNode = createNode(value);
    newNode->next = previous->next;
    previous->next = newNode;
    return head;
}
```

The loop counter describes the position occupied by `previous`. The loop ends when `previous` reaches position `position - 1`, or when the list ends too soon.

For a list of length `n`, a valid insertion position lies between 1 and `n + 1`. The worst-case time is `O(n)` and the additional space is `O(1)`.

> **Change the wording, change the traversal**
> How would the function differ if Mission Control said, "Insert *after* the fiftieth stop"? State the exact node at which your traversal must stop before writing any code.

6. The Element Hidden Somewhere Along the Trail

The mining council did not know the planet's name.

They knew only that one of the bot's spectrometers had recorded a rare element called **Aetherium**. The old transmission containing the location had been corrupted, but the bot had written the detailed readings into the appropriate beacon.

The answer was somewhere along the route.

A mining team was sent to find it.

They received `head`, entered at the first planet, inspected its recorded data, and followed `next` when Aetherium was absent. They repeated the process at every beacon.

There was no clever coordinate calculation to perform. The physical positions of the planets did not encode the logical order of the mission.

> **Pause at Mission Control**
> What two events can end this search? And if the team finds the element, what should the program return: the data, the position, or the address of the beacon itself?

One event is success: the current beacon contains the required reading.

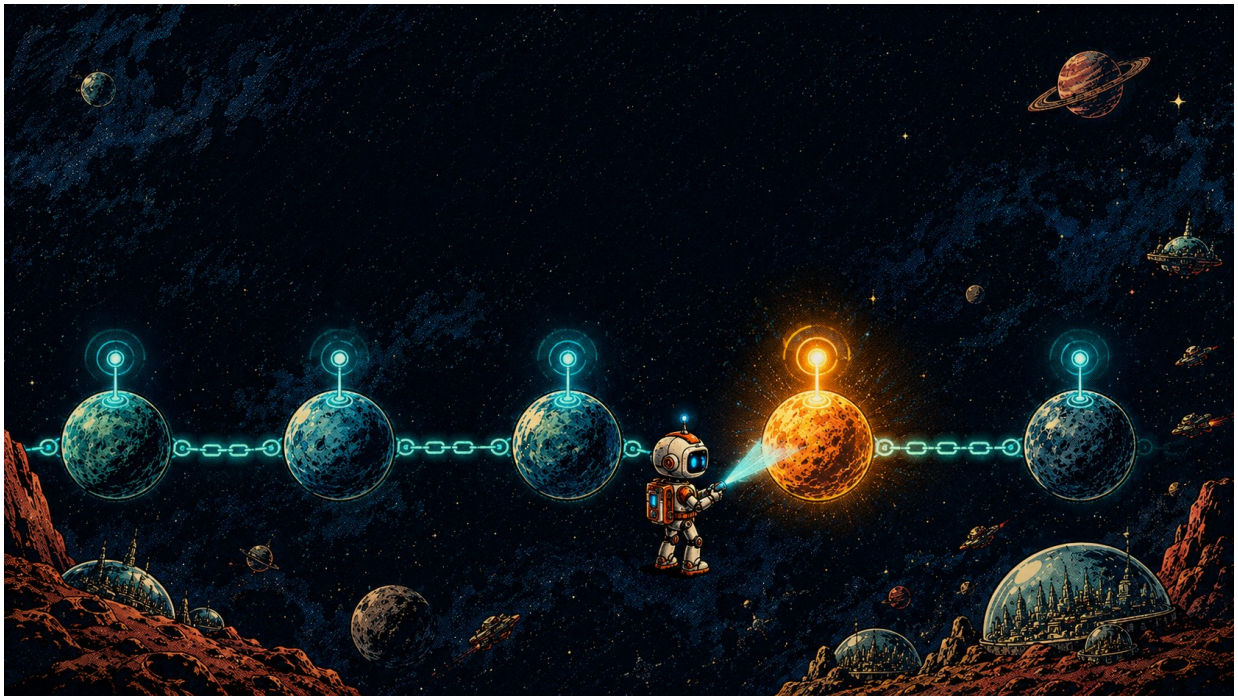
The other is exhaustion: `current` becomes `NULL`, proving that every reachable beacon was inspected without a match.

Returning the matching node's address is often the most useful choice. The caller may inspect the data, insert something after the node, or begin another operation from that location. Returning `NULL` naturally means no matching node was found.

We must remember that the returned pointer is not a permanent certificate of ownership. If the node is later deleted, the saved pointer becomes dangling.

> **What have we just discovered?**
> The mining team has performed a **linear search in a singly linked list** by following the route from `head` until a match or `NULL` is reached.

Computing Translation



Search follows the logical chain one beacon at a time.

```
Node *findNode(Node *head, int key)
{
    Node *current = head;

    while (current != NULL && current->data != key)
        current = current->next;

    return current;
}
```

If the key is present, `current` points to its first occurrence. If the key is absent, the loop ends with `current == NULL`.

The worst-case time is `O(n)`. The function uses `O(1)` additional space.

> **The council changes its request**
> What if Aetherium may occur on several planets and the mining council wants every matching location? Would returning one pointer still be enough? Could you process the results during one traversal without building another list?

7. When a World Must Disappear

The warning arrived from Virelia.

Its beacon had detected a gravitational distortion that did not belong there. Over the next six hours the distortion grew, bending light from the surrounding stars into a thin arc.

Mission Control confirmed Caitlyn's suspicion: a wandering micro black hole was crossing the region. Virelia would soon become unreachable.

The planet had to be removed from the route.

There was no time to move the planet, obviously. Nor was there any need to erase the scientific data already stored at Mission Control. What had to change was the live chain of directions.

Caitlyn located Virelia's beacon. Its `next` field still contained the address of the safe planet beyond it.

Her first instinct was to decommission the dangerous beacon immediately.

> **Pause at Mission Control**
> If the storage occupied by Virelia's node is released now, what happens to the coordinate of the safe route beyond it? And what address remains inside the preceding beacon?

The preceding beacon would still point to Virelia's old location.

The allocator would be free to reuse that memory. Perhaps another structure would receive it. In the story, perhaps another civilization would occupy the released coordinate and install completely different information there.

A traveller following the stale address would not necessarily crash immediately. That would almost be kinder. The traveller might arrive somewhere that still looks valid but now means something entirely different.

That is the danger of a dangling pointer: the address remains, but our right to interpret the storage has ended.

There is another loss. Virelia's `next` field contains the only route from Virelia to the safe suffix. Once Virelia is freed, reading that field is no longer permitted.

So the endangered coordinate must be used before the node is released.

Suppose `previous` points to the planet before Virelia and `current` points to Virelia. The living route can bypass the danger with:

```
previous->next = current->next;
```

Only after that repair may Caitlyn release `current`.

```
free(current);
```

Repair the live route first. Release the retired node second.

The black hole crisis revealed the general rule, but Mission Control soon asked two uncomfortable boundary questions.

What if the endangered planet were Ephyra, the first node?

No `previous` node exists. Caitlyn must preserve the old `head`, move `head` to `head->next`, and then release the old first node.

What if the endangered planet were the final outpost?

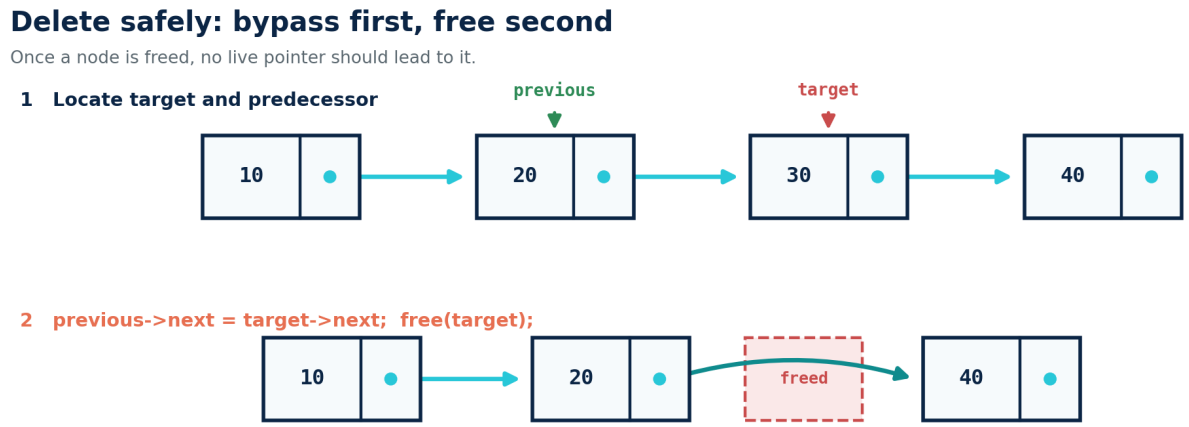
Its `next` is already `NULL`, so no suffix needs saving. Caitlyn must still find the second-last node, set its `next` to `NULL`, and release the old last node. If the route contains only one node, deleting the last node also deletes the first; the new `head` becomes `NULL`.

Three positions, one principle.

The apparent special cases come from who owns the link that must change.

> **What have we just discovered?**
> This is **deletion from a singly linked list**. Whether the target is first, internal, or last, the operation must preserve reachability, repair every live link that refers to the target, and only then release its memory.

Computing Translation



The safe route bypasses the target before the target is freed.

When the first world is unsafe

```
Node *deleteFront(Node *head)
{
    if (head == NULL) {
        printf("List is empty.\n");
        return NULL;
    }

    Node *oldHead = head;
    head = head->next;
    free(oldHead);
    return head;
}
```

Moving `head` first preserves the remaining list. The old node is released only after the caller's new entrance is known. The operation takes `O(1)` time.

When the farthest outpost is unsafe

```
Node *deleteBack(Node *head)
{
    if (head == NULL) {
        printf("List is empty.\n");
        return NULL;
    }

    if (head->next == NULL) {
        free(head);
        return NULL;
    }

    Node *previous = head;
    Node *current = head->next;

    while (current->next != NULL) {
        previous = current;
        current = current->next;
    }

    previous->next = NULL;
    free(current);
    return head;
}
```

The traversal stops with `current` at the last node and `previous` at the second-last. Finding them takes `O(n)` time.

Even a stored `tail` does not by itself make this deletion `O(1)` in a singly linked list. `tail` knows the last node, but the last node does not know its predecessor.

When an internal world is unsafe

```
Node *deleteValue(Node *head, int key)
{
    if (head == NULL)
        return NULL;

    if (head->data == key)
        return deleteFront(head);

    Node *previous = head;
    Node *current = head->next;

    while (current != NULL && current->data != key) {
        previous = current;
        current = current->next;
    }

    if (current == NULL) {
        printf("%d not found; list unchanged.\n", key);
        return head;
    }

    previous->next = current->next;
    free(current);
    return head;
}
```

The search may take `O(n)` time; the bypass and release take `O(1)`.

> **The storm does not strike only once**
> Suppose a radiation front makes every planet with a risk reading above a threshold unsafe. How

> would you delete all matching nodes in one traversal? Pay special attention to consecutive dangerous nodes and to the possibility that the first several nodes must all disappear.

8. The Arrows Caitlyn Leaves Behind

Caitlyn first noticed the scout near Kaelus.

It was too distant to identify and too careful to approach. Each time her ship changed direction, the small craft disappeared behind a moon or asteroid and reappeared later along the same route.

Perhaps it was harmless.

Caitlyn had survived enough missions to dislike plans built entirely on "perhaps."

The beacons created a perfect trail for anyone who found its entrance. Begin at the first coordinate, follow each `next`, and the route would reveal every destination the expedition had discovered.

She wanted a trail that remained useful to her but not to anyone following behind.

There was also a less dramatic concern. Her ship stored a private copy of the route, but hardware fails. If the onboard map were damaged at the edge of the expedition, the outward-facing beacons would not tell her how to return home.

Both problems suggested the same strange idea.

As Caitlyn travelled outward, she could turn each beacon after using it. Once she had read the address of the next planet, she would make the current beacon point to the planet she had just left.

Behind her, the arrows would face home.

The scout would no longer receive an outward path. If Caitlyn's onboard map failed, she could use the reversed trail to return.

The plan sounded almost too neat.

Then Caitlyn stood at the first beacon and reached for its route compartment.

> **Pause at Mission Control**
> The current beacon contains the only address of the next planet. Caitlyn wants to overwrite it with the address of the previous planet. If she turns the arrow now, how will she know where to travel next?

Suppose she writes:

```
current->next = previous;
```

The first arrow now points backward exactly as intended.

The outward address is gone.

Caitlyn has protected the trail from the scout by also protecting it from herself.

The assignment is not wrong. Its timing is wrong.

Before turning the current arrow, she must save the address that the arrow currently contains.

One pointer cannot play every role. The operation needs three distinct responsibilities:

- `previous` identifies the already reversed route behind Caitlyn;
- `current` identifies the beacon whose arrow she is turning now;
- `next` temporarily preserves the unreversed route ahead.

At each planet, Caitlyn performs the same four-part ritual.

First, save the route ahead:

```
next = current->next;
```

Second, turn the current beacon toward the route already reversed:

```
current->next = previous;
```

Third, acknowledge that the current beacon is now the front of the reversed part:

```
previous = current;
```

Finally, continue to the address that was saved before the turn:

```
current = next;
```

Save. Turn. Advance.

The two regions remain visible after every iteration. `previous` reaches the reversed prefix. `current` reaches the unreversed suffix. Neither half is lost.

When `current` finally becomes `NULL`, there is no unreversed suffix left. `previous` points to the old final node, which is now the first node of the reversed route.

That address must become the new `head`.

Caitlyn continued outward under this protection. The scout reached the route behind her and found every beacon pointing the wrong way. It could retreat toward home, but it could not follow her deeper into the expedition.

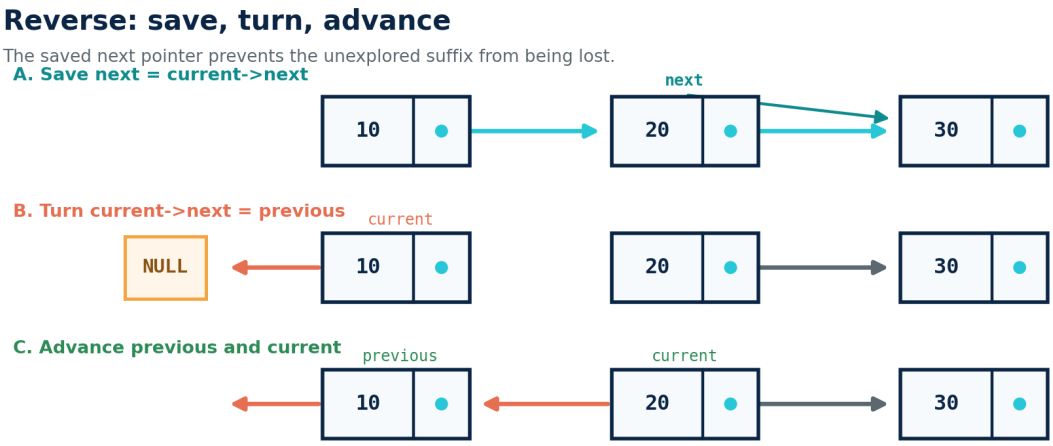
Much later, Caitlyn returned. She followed the homeward-facing arrows until she reached the original entry. Then, on her next outward trip, she performed the same operation again. Reversing a reversed list restored the original direction one beacon at a time.

The route remained one-way.

Only Caitlyn chose which way.

> **What have we just discovered?**
> Caitlyn has performed **iterative reversal of a singly linked list**. Each original `next` address is saved before the link is redirected.

Computing Translation



Reversal is a repeated save-turn-advance sequence.

```
Node *reverse(Node *head)
{
    Node *previous = NULL;
    Node *current = head;

    while (current != NULL) {
        Node *next = current->next;  /* save */
        current->next = previous;    /* turn */
        previous = current;          /* advance */
        current = next;
    }

    return previous;
}
```

Trace: `10 -> 20 -> 30 -> NULL`

Moment	previous	current	Reachable chains
Start	`NULL`	`10`	Unreversed: `10 -> 20 -> 30 -> NULL`
After 1	`10`	`20`	Reversed: `10 -> NULL`; unreversed: `20 -> 30 -> NULL`
After 2	`20`	`30`	Reversed: `20 -> 10 -> NULL`; unreversed: `30 -> NULL`
After 3	`30`	`NULL`	Reversed: `30 -> 20 -> 10 -> NULL`

Every node is visited once, so reversal takes `O(n)` time. Only a fixed number of pointers is used, giving `O(1)` additional space.

The empty list and one-node list require no special branches. The same loop handles both safely.

> **Do not run the code yet**
> Trace `previous`, `current`, and `next` for `5 -> 12 -> 18 -> 27 -> NULL`. At every iteration, draw both the reversed prefix and the unreversed suffix. If either becomes unreachable, identify the assignment that caused it.

9. A Trail Waiting at the Edge of the Map

The distant radio signal that reopened the expedition eventually led Caitlyn to a quiet red planet named **Sereph**.

Sereph was the current end of her route. Its beacon pointed to `NULL`.

On the far side of the planet, Caitlyn discovered another beacon.

It was not hers.

The markings belonged to the Aruun, the same civilization that operated Haven-9. Their explorers had started from Sereph years earlier and mapped a chain of worlds extending into another region.

The Aruun archivists were delighted that Caitlyn had found the old trail. They offered the complete route to Mission Control.

Now Caitlyn possessed two independent linked routes:

- her route from home to Sereph;
- the Aruun route from its first unexplored destination onward.

No planet needed to be copied. No beacon needed to be rebuilt.

> **Pause at Mission Control**
> Which single coordinate can turn these two independent routes into one continuous journey? What should happen if Caitlyn's first route is empty? And after joining them, may both old route handles still behave as independent owners?

The final beacon of the first route currently points to `NULL`. That is the open connection.

Caitlyn follows the first route to its last node and replaces that `NULL` with the `head` of the Aruun route.

If the first route is empty, no last beacon exists; the second route is already the complete answer.

The pointer repair is simple. Ownership is not.

After the connection, the nodes of the Aruun trail are reachable through Caitlyn's `head`. Treating the old Aruun `head` as a second independent owning list would be dangerous. If both handles later tried to free their routes, the shared nodes would be released twice.

The arrows have transferred a route even though no data moved.

> **What have we just discovered?**
> This is **concatenation of two singly linked lists**. The last node of the first list is connected to the head of the second list.

Computing Translation

```
Node *concatenate(Node *first, Node *second)
{
    if (first == NULL)
        return second;

    Node *current = first;
    while (current->next != NULL)
        current = current->next;

    current->next = second;
    return first;
}
```

With only `head`, finding the last node of `first` takes `O(n)` time, where `n` is the length of the first list. The link change itself takes `O(1)` time.

If the first list maintains a valid `tail`, concatenation can be `O(1)`. The combined route's tail becomes the second list's tail when the second list is nonempty.

After concatenation, the caller should make the ownership transfer explicit:

```
first = concatenate(first, second);
second = NULL;   /* second no longer owns a separate route */
```

> **A suspicious coincidence**
> What if the two routes already share one node farther ahead? Connecting them blindly could create a cycle or cause part of the route to be visited twice. How could Mission Control detect shared nodes before concatenation?

10. The Path That Began at Orialis

The Aruun trail at Sereph joined neatly at the end of Caitlyn's route.

The next offer arrived in a less convenient place.

During a later visit to Orialis, Caitlyn met a small survey team from the **Mireen Collective**. Their expedition also began at Orialis, travelled through three remarkable moons, and eventually returned to a planet already present after Orialis in Caitlyn's route.

The Mireen were willing to share every discovery.

Caitlyn wanted future travellers to leave Orialis, follow the complete Mireen trail, and then return to the destination that had originally followed Orialis.

This was the question Nysa had quietly left behind earlier in the chapter.

Inserting one moon required two assignments. Inserting a whole route should not require rebuilding every node. But two boundaries now matter:

- the place where the branch begins;
- the place where it returns to Caitlyn's original suffix.

> **Pause at Mission Control**
> Before Orialis is redirected, which original address must survive? Which node of the Mireen route must eventually store that address?

Orialis currently owns the address of the old successor. That coordinate must be preserved before anything overwrites `current->next`.

The first node of the Mireen route can become Orialis's new successor. But if we merely write:

```
current->next = branch;
```

the Mireen trail will run correctly until its last beacon reaches `NULL`. Caitlyn's original suffix will still be lost.

The last Mireen beacon must point back to the old successor.

So Caitlyn first finds `branchTail`, the last node of the inserted route. Then she connects it to the endangered suffix:

```
branchTail->next = current->next;
```

Only after the old route is safe does she redirect Orialis:

```
current->next = branch;
```

The same principle from Nysa has expanded, not changed.

Preserve the suffix first. Redirect the predecessor second.

The inserted route now belongs to the combined expedition. Its old handle must no longer be treated as an independent owner.

> **What have we just discovered?**
> This is **splicing one singly linked list into the middle of another**. The inserted list's tail preserves the old suffix before the predecessor is redirected to the inserted head.

Computing Translation

```
Node *spliceAfter(Node *head, int key, Node *branch)
{
    if (branch == NULL)
        return head;

    Node *current = findNode(head, key);
    if (current == NULL) {
        printf("%d not found; routes unchanged.\n", key);
        return head;
    }

    Node *branchTail = branch;
    while (branchTail->next != NULL)
        branchTail = branchTail->next;

    branchTail->next = current->next;
    current->next = branch;
    return head;
}
```

Let the main list contain `n` nodes and the branch contain `m` nodes. Searching for `key` may take `O(n)` time and finding the branch tail may take `O(m)`, so the total is `O(n + m)` with only head pointers.

If the caller already knows `current` and the branch maintains `tail`, the splice itself takes `O(1)`.

The function assumes that the two lists are disjoint and acyclic. If the branch already contains a node from the main route, splicing may create a cycle. Ownership and structural preconditions are part of correctness even when the C type system cannot express them.

> **Detach the shared expedition**
> Months later the Mireen want their three-moon route returned as an independent list. Which two addresses would Caitlyn need to remember in order to detach it cleanly and reconnect the original route?

11. When the Beacons Finally Fall Silent

Years after the first signal, the expedition was formally closed.

The discoveries were copied into permanent archives. The planets remained, of course, but the temporary beacons no longer needed to occupy Mission Control's allocation network.

Caitlyn opened the route one final time.

The simplest way to make the map appear empty was obvious:

```
head = NULL;
```

No beacon would be reachable from `head` afterward.

That was precisely the problem.

> **Pause at Mission Control**
> If the allocated nodes still exist but their only entrance is erased, who can release them now? Does making a route unreachable return its memory to the allocator?

It does not.

Changing `head` changes one pointer. It does not visit or free any node. The beacons become unreachable allocated storage: a memory leak.

Caitlyn must dismantle them one at a time.

At each node, another familiar danger appears. The current node contains the only route to the next node. If she frees the current node first, reading `current->next` afterward is forbidden.

So she saves the next address, frees the current node, and advances to the saved address.

The process is deletion from the beginning repeated until the route is empty.

When the loop ends, `head` is `NULL` not because the entrance was thrown away, but because every allocation reachable through it was properly released.

> **What have we just discovered?**
> This is **clearing or destroying a singly linked list**. Each next address is saved before its node is freed, preventing both memory leaks and use-after-free.

Computing Translation

```
Node *clearList(Node *head)
{
    while (head != NULL) {
        Node *next = head->next;
        free(head);
        head = next;
    }

    return NULL;
}
```

The caller records the empty result:

```
head = clearList(head);
```

Clearing `n` nodes takes `O(n)` time and `O(1)` additional space.

Any other pointer that still refers to one of the released nodes becomes dangling. The program must know who owns the list and who is merely observing it. C will not enforce that agreement for us.

Mission Control Reference Desk

The story discovered the operations before naming them. This section gathers the common technical foundation in one place.

The node and its allocator

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *createNode(int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
```

Displaying the route without moving head

```
void display(const Node *head)
{
    const Node *current = head;

    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }

    printf("NULL\n");
}
```

A small mission log

The following driver does not replace the individual explanations. It shows how returned-head functions are used together.

```
int main(void)
{
    Node *head = NULL;

    head = insertBack(head, 10);
    head = insertBack(head, 20);
    head = insertFront(head, 5);
    head = insertAfter(head, 10, 15);
    head = insertBefore(head, 20, 18);
    head = insertAt(head, 3, 8);

    display(head);

    Node *found = findNode(head, 18);
    printf("18 %s found.\n", found != NULL ? "was" : "was not");

    head = deleteValue(head, 15);
    head = reverse(head);
    display(head);

    head = clearList(head);
    return 0;
}
```

Technical Detour: Letting a Function Change `head` Directly

Throughout Caitlyn's main journey, an operation that changes the first node returns the updated `head`:

```
head = insertFront(head, 12);
head = deleteFront(head);
head = reverse(head);
```

This convention keeps the pointer flow visible at the call site. C also permits another interface: pass the address of the caller's `head` pointer.

```
void insertFrontPP(Node **head, int value)
{
    Node *newNode = createNode(value);
    newNode->next = *head;
    *head = newNode;
}

/* caller */
insertFrontPP(&head, 12);
```

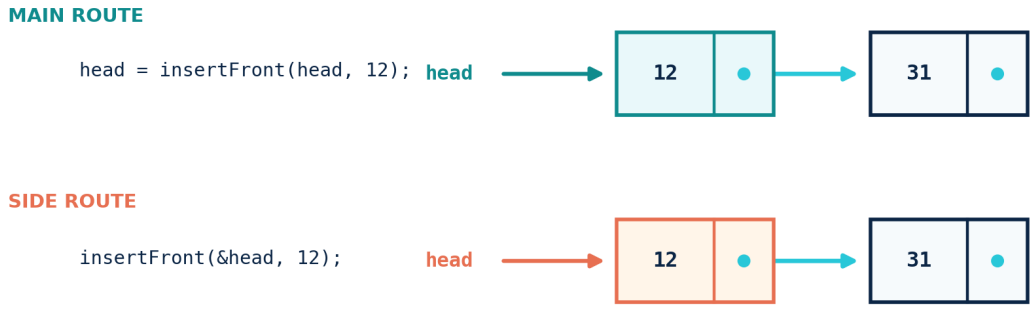
Read the levels carefully:

- `head` in the caller is a `Node *` that stores the first node's address.
- `&head` is the address of that pointer variable.
- the parameter `Node **head` receives that address;
- `*head` therefore refers to the caller's actual `Node *`.

The same approach can implement deletion, reversal, and any operation that may replace the first node. Neither convention is universally superior. A program should choose one clear interface and use it consistently.

Two ways to let a function change head

Main route: return the updated head. Side route: pass the address of head.



Both are valid C. This chapter uses the returned-head version for the main journey.

Returned-head and double-pointer interfaces are both valid; this journey uses returned heads.

Missions to Ponder Before Coding

These are not syntax exercises. For each event, first solve the mission problem, then translate it into nodes and links.

1. The radiation corridor

A radiation front makes every planet with a risk value above 80 unsafe. Several unsafe planets may occur consecutively, including at the beginning and end of the route.

Which pointer identifies the previous live planet? When a node is removed, should that pointer advance?

2. Two trails that secretly meet

Caitlyn wants to concatenate two friendly routes, but both routes already pass through Orialis.

What structure results if she joins them without detecting the shared suffix? How could pointer identity reveal the intersection?

3. The temporary Mireen detour

The Mireen route is spliced after Orialis for one survey season. Later it must be detached and returned as an independent list while Caitlyn's original route is restored.

Which two boundary links must be remembered?

4. The moving service station

The station before stop 50 must be moved so that it appears before stop 20. You may not copy the node or allocate a replacement.

How can the existing node be detached and reinserted without freeing it?

5. A route divided between two teams

One team needs the first half of the route and another team needs the second half. The original nodes must be reused.

How could a slow pointer and a fast pointer locate the dividing point in one traversal?

6. The beacon that belongs to nobody

A programmer allocates a new node and then discovers that the requested insertion position is invalid.

Who owns the new allocation? What must happen before the function returns?

7. Reversal under interruption

Caitlyn has reversed only the first four links when communication fails.

How many separate reachable chains exist at that moment? Which pointers are required to describe all of them?

The Question the One-Way Route Cannot Avoid

Caitlyn's route has survived expansion, detours, danger, combination, reversal, and retirement.

Its strength comes from a wonderfully small promise: each beacon knows the destination that follows it.

That promise is also its limitation.

When Caitlyn needed to travel home, she reversed every link. It worked. But should every traveller have to rebuild the route merely to move backward?

What if each beacon remembered not only where the journey continues, but also where it came from?

And another mission creates a different question. Some expeditions are not journeys toward a final destination. A patrol may need to visit the same worlds repeatedly, beginning again after the last stop.

What if the final beacon did not say `NULL`?

What if it quietly pointed home?

Those are not repairs to Caitlyn's current route.

They are new ways for a linked route to exist.